

# COMP 3311

# DATABASE MANAGEMENT

# SYSTEMS

## LECTURE 20 EXERCISES

### QUERY PROCESSING: COST ESTIMATES

## EXERCISE 1

Use the following information to estimate the page I/O cost to compute the query result and the number of query result pages for the query.

```
select distinct sName  
from Sailor;
```

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

The Sailor relation consists of 60,000 tuples sorted on sailor id.

The tuple size is 26 bytes.

The page size is 1200 bytes so  $bf_{\text{Sailor}} = \lfloor 1200 / 26 \rfloor = \lfloor 46.15 \rfloor = 46$ .

Therefore,  $B_{\text{Sailor}} = \lceil 60000 / 46 \rceil = \lceil 1,304.34 \rceil = 1,305$  pages.

The buffer size  $M$  is 250 pages and 4% of sailor names are the same.

There is no index.

```
select distinct sName
from Sailor;
```

$n_{\text{Sailor}}$ : 60,000  
 $bf_{\text{Sailor}}$ : 46 tuples/page  
 $B_{\text{Sailor}}$ : 1,305 pages  
 Page size: 1,200 bytes  
 $M$  buffer pages: 250

## EXERCISE 1 (CONTD)

**Sort:** For each sorted run read up to 250 pages containing up to 11,500 tuples/run. Keep only sName (10 bytes).  $bf_{sName} = \lfloor 1200/10 \rfloor = 120$  tuples. Create totally  $\lceil 1305/250 \rceil = 6$  sorted runs. Write  $\lceil 11500/120 \rceil = 96$  pages for 5 sorted runs and  $\lceil 2500/120 \rceil = 21$  pages for 1 sorted run. Totally write  $(96 \cdot 5) + 21 = 501$  pages.

Sort page I/O cost: 1305 pages read + 501 pages written = 1806.

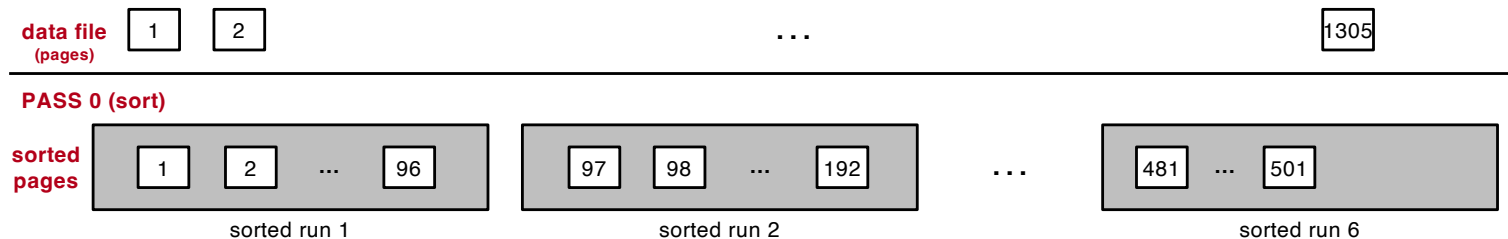
**Merge:** Read and merge the 501 sorted pages using a 6-way merge.

Merge page I/O cost: 501

Projection page I/O cost:  $1806 + 501 = 2307$

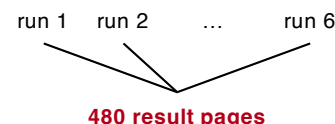
**Recall:** The cost to write the query result is never included in the cost estimates.

Query result pages:  $\lceil 60000 \text{ tuples} \cdot .96 / \lfloor 1200/10 \rfloor \text{ tuples/page} \rceil = 480$  since it is assumed that 4% of sailor names are the same.



Since there are 250 buffer pages, for each sorting run read into the buffer up to 250 Sailor pages but write up to  $\lceil 11500 \text{ tuples} / \lfloor 1200/10 \text{ tuples/page} \rfloor \rceil = 96$  pages since only sName is kept as the other attributes are not needed. There are  $\lceil 1305/250 \rceil = 6$  full sorted runs created each of size 96 pages and 1 sorted run of  $\lceil 2500 \text{ tuples} / \lfloor 1200/10 \text{ tuples/page} \rfloor \rceil = 21$  pages.

**PASS 1 (merge)**



## EXERCISE 2

Use the following information about the relations to estimate the page I/O cost to compute the query result using the stated join strategies.

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

Reserves(sailorId: 2 bytes, boatId: 2 bytes, rDate: 7 bytes)

**Query:** `select *  
from Sailor natural join Reserves;`

Page size: 500 bytes; buffer size  $M$ : 100 pages; pointer size: 4 bytes.

Each Sailor tuple is 26 bytes, and each Reserves tuple is 11 bytes.

$n_{\text{Sailor}} = 20,000$  tuples;  $bf_{\text{Sailor}} = \lfloor 500 / 26 \rfloor = 19$  Sailor tuples per page.

$n_{\text{Reserves}} = 60,000$  tuples;  $bf_{\text{Reserves}} = \lfloor 500 / 11 \rfloor = 45$  Reserves tuples per page.

$B_{\text{Sailor}} = \lceil 20,000 / 19 \rceil = 1,053$  pages;  $B_{\text{Reserves}} = \lceil 60,000 / 45 \rceil = 1,334$  pages.

Since there are 20,000 Sailor tuples and 60,000 Reserves tuples, a sailor has, on average, 3 reservations.

```
select *  
from Sailor natural join Reserves;
```

$n_{\text{Sailor}}$ : 20,000  
 $bf_{\text{Sailor}}$ : 19  
 $B_{\text{Sailor}}$ : 1,053  
 $n_{\text{Reserves}}$ : 60,000  
 $bf_{\text{Reserves}}$ : 45  
 $B_{\text{Reserves}}$ : 1,334  
 $M$  buffer pages: 100  
Reservations/sailor: 3

## EXERCISE 2 (CONT'D)

a) optimized block nested-loop join

i. using Sailor as the outer relation:  $\lceil B_r / (M-2) \rceil * B_s + B_r$

Read 98 pages of the Sailor relation into the buffer at a time (there are  $\lceil 1,053 / 98 \rceil = \lceil 10.74 \rceil = 11$  “blocks” of Sailor pages). (*Requires 98 buffer pages*)

For each “block” of Sailor pages we scan the Reserves relation (page-by-page) to find matching tuples. (*Requires 1 buffer page*)

Write matching results to the output. (*Requires 1 buffer page*)

Query page I/O cost:  $11 * 1,334 + 1,053 = 15,727$

ii. using Reserves as the outer relation:  $\lceil B_r / (M-2) \rceil * B_s + B_r$

Read 98 pages of the Reserves relation into the buffer at a time (there are  $\lceil 1,334 / 98 \rceil = \lceil 13.71 \rceil = 14$  “blocks” of Reserves pages). (*Requires 98 buffer pages*)

For each “block” of Reserves pages we scan the Sailor relation (page-by-page) to find matching tuples. (*Requires 1 buffer page*)

Write matching results to the output. (*Requires 1 buffer page*)

Query page I/O cost:  $14 * 1,053 + 1,334 = 16,076$

```
select *
from Sailor natural join Reserves;
```

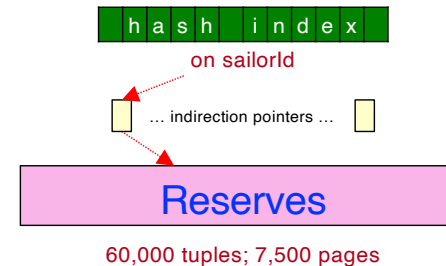
## EXERCISE 2 (CONT'D)

$n_{\text{Sailor}}: 20,000$   
 $bf_{\text{Sailor}}: 19$   
 $B_{\text{Sailor}}: 1,053$   
 $n_{\text{Reserves}}: 60,000$   
 $bf_{\text{Reserves}}: 45$   
 $B_{\text{Reserves}}: 1,334$   
 $M_{\text{buffer pages}}: 100$   
 $\text{Reservations/sailor}: 3$

b) indexed nested-loop join with hash index on Reserves.sailorId (assume no overflow).

For each Sailor tuple, find the corresponding entry in the hash index on Reserves.sailorId.

This takes 1 page I/O per Sailor tuple (since there is no overflow).



Since each Sailor tuple has on average 3 reservations and, since the hash index is non-clustering (secondary), each sailorId is expected to have 3 matching tuples in the Reserves relation. Thus, there is an indirection level, and 1 page I/O per sailorId is needed to retrieve the indirection pointers.

These 3 indirection pointers occupy  $(3 \text{ pointers} * 4 \text{ bytes per pointer}) = 12 \text{ bytes}$ , which easily fit on one page.

Finally, 3 page I/Os per sailorId are needed to retrieve the corresponding Reserves records (i.e., 1 page I/O for each indirection pointer).

Query page I/O cost: cost of reading Sailor + (# tuples in Sailor \* (1 for hash index + 1 for indirection pointers + 3 for Reserves records))  
 $= 1053 + 20,000 * 5 = 101,053$

```
select *
from Sailor natural join Reserves;
```

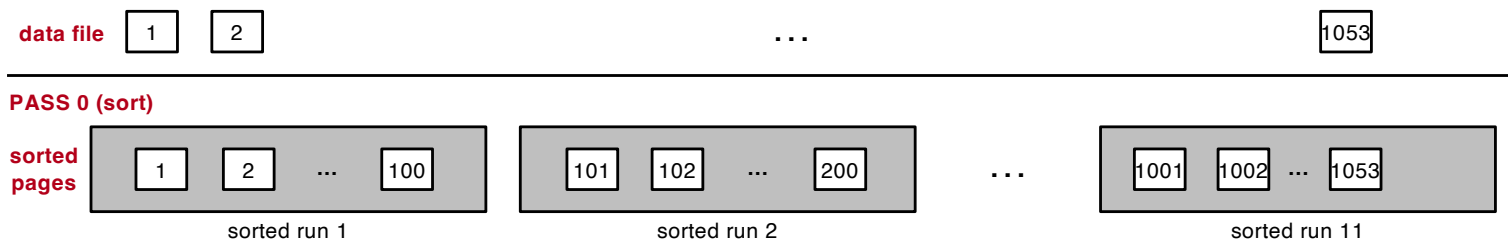
$n_{\text{Sailor}}: 20,000$   
 $bf_{\text{Sailor}}: 19$   
 $B_{\text{Sailor}}: 1,053$   
 $n_{\text{Reserves}}: 60,000$   
 $bf_{\text{Reserves}}: 45$   
 $B_{\text{Reserves}}: 1,334$   
 $M_{\text{buffer pages}}: 100$   
 $\text{Reservations/sailor}: 3$

## EXERCISE 2 (CONT'D)

c) merge join (assume that both relations are not sorted initially)

Sort cost (to sort the relations using external sorting)

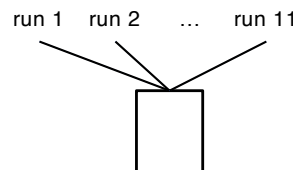
Cost to sort Sailor on sailord



Since there are 100 buffer pages, for each sorting run read into the buffer 100 Sailor pages and write 100 pages. There are  $\lceil 1053/100 \rceil = 11$  sorted runs created, 10 of size 100 pages and 1 of size 53 pages. Totally  $1053 \times 2 = 2106$  pages are read and written.

PASS 1 (merge)

The Sailor relation can be merged in 1 pass since only 11 buffer pages are required to merge the 11 sorted runs. Totally  $1053 \times 2 = 2106$  pages are read and written.



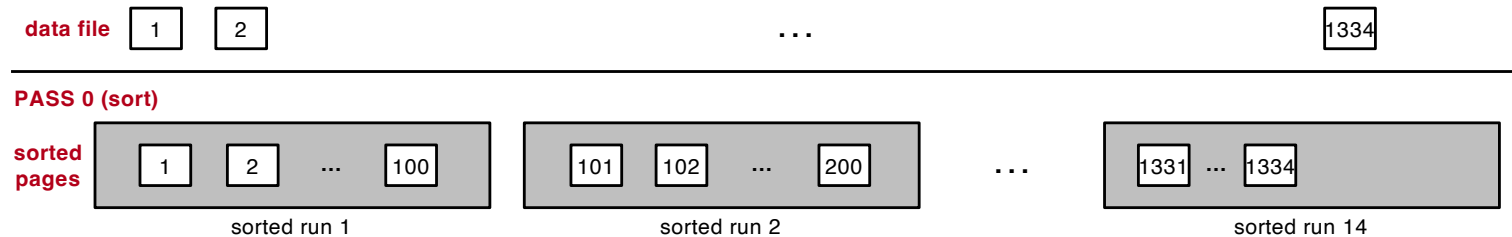
Page I/O cost to sort Sailor:  $1053_r + 1053_w$  (Pass 0) +  $1053_r + 1053_w$  (Pass 1) = 4,212

```
select *
from Sailor natural join Reserves;
```

## EXERCISE 2 (CONT'D)

$n_{\text{Sailor}}: 20,000$   
 $bf_{\text{Sailor}}: 19$   
 $B_{\text{Sailor}}: 1,053$   
 $n_{\text{Reserves}}: 60,000$   
 $bf_{\text{Reserves}}: 45$   
 $B_{\text{Reserves}}: 1,334$   
 $M_{\text{buffer pages}}: 100$   
 $\text{Reservations/sailor}: 3$

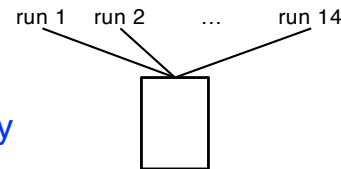
### Cost to sort Reserves on sailord



Since there are 100 buffer pages, for each sorting run read into the buffer 100 Reserves pages and write 100 pages. There are  $\lceil 1334/100 \rceil = 14$  sorted runs created 13 of size 100 pages and 1 of size 34 pages. Totally  $1334 \times 2 = 2668$  pages are read and written.

#### PASS 1 (merge)

The Reserves relation can be merged in 1 pass since only 14 buffer pages are required to merge the 14 sorted runs. Totally  $1334 \times 2 = 2668$  pages are read and written.



Page I/O cost to sort Reserves:  $\underset{r}{1334} + \underset{w}{1334}$  (Pass 0) +  $\underset{r}{1334} + \underset{w}{1334}$  (Pass 1) = 5,336

Total page I/O cost to sort the relations:  $4212 + 5336 = \underline{9,548}$

Merge cost (to join the relations)

Read both sorted relations and join during merge.

Total page I/O cost to join the relations:  $1053 + 1334 = \underline{2,387}$

Query page I/O cost:  $9548 + 2387 = \underline{11,935}$



```
select *  
from Sailor natural join Reserves;
```

$n_{\text{Sailor}}$ : 20,000  
 $bf_{\text{Sailor}}$ : 19  
 $B_{\text{Sailor}}$ : 1,053  
 $n_{\text{Reserves}}$ : 60,000  
 $bf_{\text{Reserves}}$ : 45  
 $B_{\text{Reserves}}$ : 1,334  
 $M$  buffer pages: 100  
Reservations/sailor: 3

## EXERCISE 2 (CONT'D)

### d) hash join (assume no overflow)

Use the smaller relation ( $\text{Sailor} = 1,053$  pages) as the **build input**.

We should choose the number of partitions  $n$  such that, when doing the **join phase**, all the pages of **each build partition** of **Sailor fits in the buffer**. For example, we can use **11** partitions so that the size of each partition is  $\lceil 1053/11 \rceil = 96$  pages, which fit into the buffer.

First read and partition **Sailor** using 11 buffer pages, one for each partition.

**Partition Sailor page I/O cost:**  $1053 + 1053 = \underline{2,106}$ .

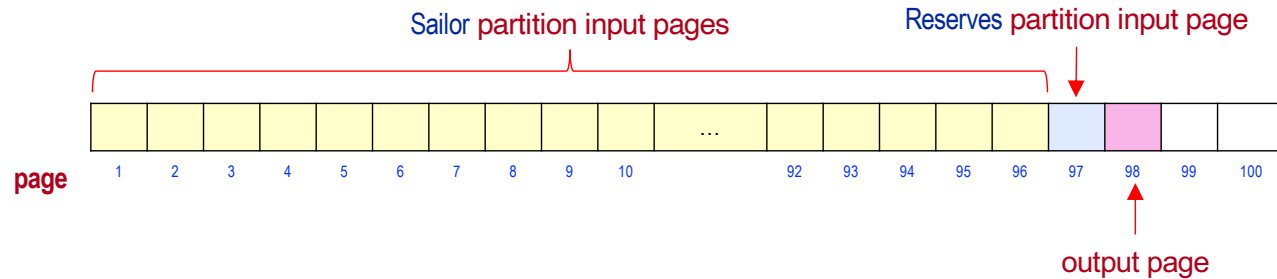
Then read and partition **Reserves** using 11 buffer pages, one for each partition.

**Partition Reserves page I/O cost:**  $1334 + 1334 = \underline{2,668}$ .

Note that each partition of **Reserves** occupies  $\lceil 1334/11 \rceil = 122$  pages, which is more than the available buffer of **100** pages. However, we don't care since only the partitions of **Sailor** need to fit in the buffer.

```
select *
from Sailor natural join Reserves;
```

## EXERCISE 2 (CONT'D)



$n_{\text{Sailor}}: 20,000$   
 $bf_{\text{Sailor}}: 19$   
 $B_{\text{Sailor}}: 1,053$   
 $n_{\text{Reserves}}: 60,000$   
 $bf_{\text{Reserves}}: 45$   
 $B_{\text{Reserves}}: 1,334$   
 $M_{\text{buffer pages}}: 100$   
 $\text{Reservations/sailor}: 3$

Finally, we read, in turn, each partition (i.e., 96 pages) of **Sailor**, read the corresponding partition of **Reserves** page by page, match it (i.e., by hashing) against the **Sailor** partition and output the matching tuples.

Join page I/O cost:  $1053 + 1334 = \underline{2,387}$

Query page I/O cost:  $3 * (1053 + 1334) = \underline{7,161}$

## EXERCISE 3

Use the following information about the relations to process the query.

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

Reserves(sailorId: 2 bytes, boatId: 2 bytes, rDate: 7 bytes)

The Sailor relation contains 40,000 tuples in 262 pages and the Reserves relation contains 100,000 tuples in 276 pages. There are 10 different sailor ratings and 100 different boats. Assume that sailors are distributed uniformly over the 10 ratings and reservations are distributed uniformly over the 100 boats. The page size is 4000 bytes.

**Query:**

```
select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;
```

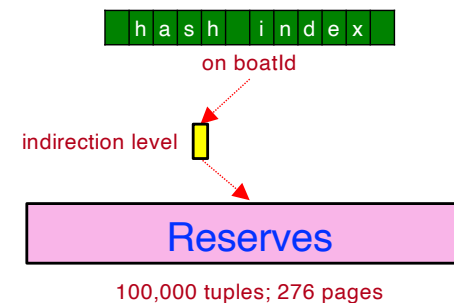
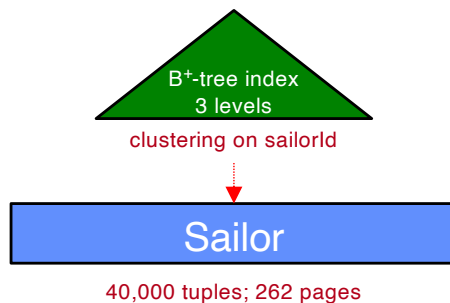
### Some useful statistics

- On average, each sailor has  $(100,000/40,000) = 2.5$  reservations.
- On average, each boat has  $(100,000/100) = 1,000$  reservations.
- On average, for each rating there are  $(40,000/10) = 4,000$  sailors.

## EXERCISE 3 (CONTD)

Size sailorId: 2 bytes  
 Size sName: 10 bytes  
 $n_{\text{Sailor}}$ : 40,000  
 $B_{\text{Sailor}}$ : 262 pages  
 $n_{\text{Reserves}}$ : 100,000  
 $B_{\text{Reserves}}$ : 276 pages  
 Record pointers/page: 400  
 Number of ratings: 10  
 Number of boats: 100  
 Page size: 4,000

The Sailor relation contains a clustering B<sup>+</sup>-tree index with 3 levels on sailorId and the Reserves relation contains a hash index on boatId. A page can fit 400 record pointers. For non-clustering indexes, each record pointer is assumed to point to a different page.



```

select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;
  
```

### Some useful statistics

- On average, each sailor has 2.5 reservations.
- On average, each boat has 1,000 reservations.
- On average, for each rating there are 4,000 sailors.

Estimate the minimum page I/O cost of processing the query using a fully pipelined execution method (i.e., do not materialize anything except the query result).

```

select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;

```

## EXERCISE 3 (CONTD)

Size sailorId: 2 bytes  
 Size sName: 10 bytes  
 $n_{\text{Sailor}}$ : 40,000  
 $B_{\text{Sailor}}$ : 262 pages  
 $n_{\text{Reserves}}$ : 100,000  
 $B_{\text{Reserves}}$ : 276 pages  
 Record pointers/page: 400  
 Number of ratings: 10  
 Number of boats: 100  
 Page size: 4,000

### Base Strategy: block nested-loop

Do not use any indexes, join **Sailor** and **Reserves** using block nested-loop and check the conditions **boatId=30** and **rating>5** in memory. Since the buffer size is unknown, assume the minimum buffer size (3 pages) for this strategy and so use the worst-case join cost.

Page I/O cost:  $262 * 276 + 262 = \underline{72,574}$

### a) Cost to evaluate $\sigma_{\text{boatId}=30} \text{Reserves}$

#### Strategy a1: linear search on Reserves

Read the **Reserves** relation page-by-page requiring 276 page I/Os and check the condition **boatId=30** in each **Reserves** tuple.

Since each boat has, on average, 1000 reservations, 1000 **Reserves** tuples will meet the condition.

Keep only **sailorId** as the result which will occupy  $\lceil 1,000 / \lfloor 4000/2 \rfloor \rceil = 1$  page.

Pipeline only the **sailorId** for each tuple to the next operation.

Page I/O cost: 276

```

select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;

```

## EXERCISE 3 (CONTD)

Size sailorId: 2 bytes  
 Size sName: 10 bytes  
 $n_{\text{Sailor}}: 40,000$   
 $B_{\text{Sailor}}: 262$  pages  
 $n_{\text{Reserves}}: 100,000$   
 $B_{\text{Reserves}}: 276$  pages  
 Record pointers/page: 400  
 Number of ratings: 10  
 Number of boats: 100  
 Page size: 4,000

**Strategy a2:** index lookup using hash index on Reserves.boatId

Hash boat id 30 requiring 1 page I/O to access the index.

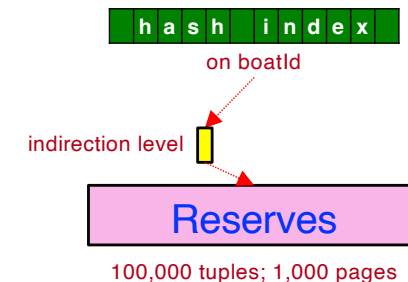
Since the 100,000 reservations are uniformly distributed over the 100 boats,  $\lceil 100,000/100 \rceil = 1000$  record pointers are returned. These 1000 record pointers occupy  $\lceil 1000/400 \rceil = 3$  pages requiring 3 page I/Os.

For each record pointer, retrieve the corresponding Reserves tuple requiring 1000 page I/Os (one for each record pointer).

Keep only sailorId as the result which will occupy  $\lceil 1,000 \lfloor 4000/2 \rfloor \rceil = 1$  page.

Pipeline only the sailorId for each tuple to the next operation.

Page I/O cost:  $1 + 3 + 1000 = \underline{1004}$



```

select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;

```

## EXERCISE 3 (CONT'D)

Size sailorId: 2 bytes  
 Size sName: 10 bytes  
 $n_{\text{Sailor}}$ : 40,000  
 $B_{\text{Sailor}}$ : 262 pages  
 $n_{\text{Reserves}}$ : 100,000  
 $B_{\text{Reserves}}$ : 276 pages  
 Record pointers/page: 400  
 Number of ratings: 10  
 Number of boats: 100  
 Page size: 4,000

b) Cost to evaluate  $\sigma_{\text{rating}>5}\text{Sailor}$  (no index on rating)

Strategy b1: linear search on Sailor

Read the **Sailor** relation page-by-page and check the condition  $\text{rating}>5$  in each tuple requiring 262 page I/Os.

Since the 10 different rating values are uniformly distributed over the **Sailor** tuples, there are 5 rating values greater than 5. Hence,  $5/10$  or  $\lceil 40,000 * 5/10 \rceil = 20,000$  **Sailor** tuples are expected to meet the condition.

Keep only **sailorId** and **sName** as the result which will occupy  $\lceil 20,000 / \lfloor 4000/12 \rfloor \rceil = 61$  pages.

Pipeline only **sailorId** and **sName** to the next operation.

Page I/O cost: 262

Strategy b2: check the condition in the buffer (*on-the-fly*) during the join

The condition  $\text{rating}>5$  is checked in the buffer for each result tuple of the join operation.

Page I/O cost: 0

```
select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;
```

## EXERCISE 3 (CONTD)

Size sailorId: 2 bytes  
Size sName: 10 bytes  
 $n_{\text{Sailor}}$ : 40,000  
 $B_{\text{Sailor}}$ : 262 pages  
 $n_{\text{Reserves}}$ : 100,000  
 $B_{\text{Reserves}}$ : 276 pages  
Record pointers/page: 400  
Number of ratings: 10  
Number of boats: 100  
Page size: 4,000

### c) Cost to evaluate Sailor ⋈ Reserves

Strategy c1: block nested-loop (following Strategy b1 – linear search on Sailor)

As the **Sailor** relation is read using **Strategy b1** and result tuples are generated, join them with the result of a) which is in the buffer.

Keep only **sName**, which is the query result.

Page I/O cost: 0

Strategy c2: block nested-loop (following Strategy b2 – check the condition in the buffer (*on-the-fly*) during the join)

Read the **Sailor** relation page-by-page requiring 262 page I/Os and join with the result of a), which is in the buffer.

Check the condition **rating**>5 in each result tuple in the buffer.

Keep only **sName**, which is the query result.

Page I/O cost: 262



```

select sName
from Sailor natural join Reserves
where boatId=30
and rating>5;

```

## EXERCISE 3 (CONT'D)

Size sailorId: 2 bytes  
 Size sName: 10 bytes  
 $n_{\text{Sailor}}$ : 40,000  
 $B_{\text{Sailor}}$ : 262 pages  
 $n_{\text{Reserves}}$ : 100,000  
 $B_{\text{Reserves}}$ : 276 pages  
 Record pointers/page: 400  
 Number of ratings: 10  
 Number of boats: 100  
 Page size: 4,000

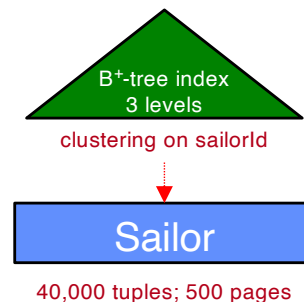
**Strategy c3:** indexed nested-loop using sailorId index on Sailor  
 (following Strategy b2 – check the condition in the buffer  
 (*on-the-fly*) after the join)

For each of the 1000 sailor ids in the result of a), use the B<sup>+</sup>-tree index on sailorId to find the corresponding Sailor tuple. The cost is 1000 \* (3 levels + 1 for record retrieval) 4000 page I/Os.

Check the condition rating>5 in each result tuple in the buffer.

Keep only sName, which is the query result.

Page I/O cost: 4000



d) Cost to evaluate  $\pi_{\text{sName}}$

**Strategy d1:** perform the projection in the buffer (*on-the-fly*) during the join

The projection is done in the buffer for each result tuple of the join operation.

Page I/O cost: 0

## EXERCISE 3 (CONTD)

### Minimum query processing page I/O cost

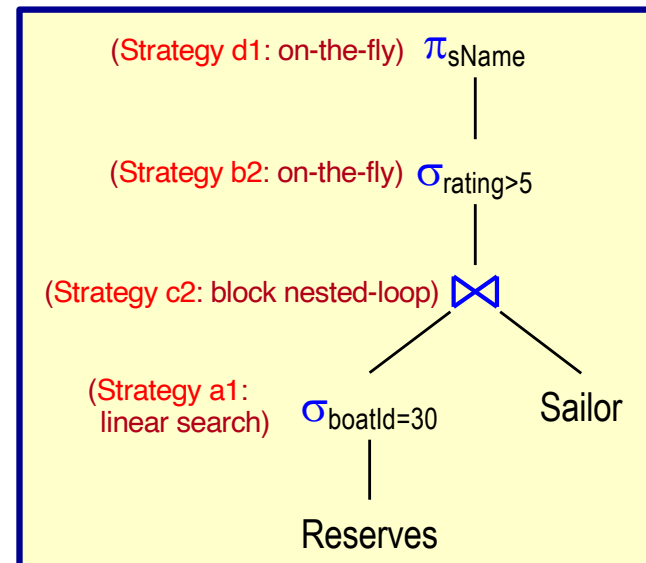
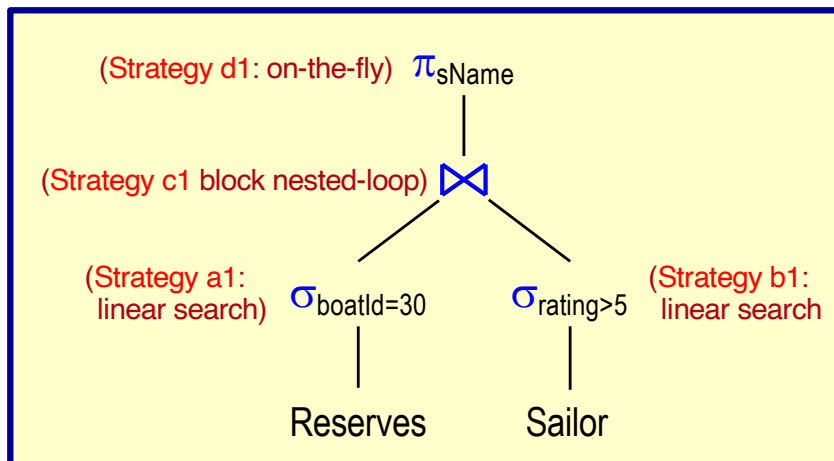
|                                                                             |   |                   |
|-----------------------------------------------------------------------------|---|-------------------|
| Strategy a1: linear search                                                  | = | 276               |
| Strategy b1: linear search                                                  | = | 262               |
| Strategy c1: block nested-loop                                              | = | 0                 |
| Strategy d1: on-the fly                                                     | = | 0                 |
| <b>Total</b>                                                                |   | <b><u>538</u></b> |
| Requires maximum 61 buffer pages for intermediate results (in Strategy b1). |   |                   |

### Minimum query processing page I/O cost

|                                                          |   |                   |
|----------------------------------------------------------|---|-------------------|
| Strategy a1: linear search                               | = | 276               |
| Strategy b2: on-the-fly                                  | = | 0                 |
| Strategy c2: block nested-loop                           | = | 262               |
| Strategy d1: on-the fly                                  | = | 0                 |
| <b>Total</b>                                             |   | <b><u>538</u></b> |
| Requires maximum 1 buffer page for intermediate results. |   |                   |

**Recall:** The cost to write the query result is never included in the cost estimate.

### Minimum Query Processing Page I/O Cost Query Evaluation Trees



## EXERCISE 4

Use the following information to estimate the page I/O cost to compute the query result and the number of query result pages for the query.

```
select distinct sName  
from Sailor;
```

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

The Sailor relation consists of 40,000 tuples sorted on sailor id.

The tuple size is 26 bytes.

The page size is 800 bytes so  $bf_{\text{Sailor}} = \lfloor 800 / 26 \rfloor = \lfloor 30.76 \rfloor = 30$ .

Therefore,  $B_{\text{Sailor}} = \lceil 40000 / 30 \rceil = \lceil 1,333.33 \rceil = 1,334$  pages.

The buffer size  $M$  is 100 pages and 5% of sailor names are the same.

There is no index.

```
select distinct sName
from Sailor;
```

$n_{\text{Sailor}}: 40,000$   
 $bf_{\text{Sailor}}: 30$   
 $B_{\text{Sailor}}: 1,334$   
 Page size: 800 bytes  
 Buffer pages  $M: 100$

## EXERCISE 4 (CONT'D)

**Sort:** For each sorted run read up to 100 pages containing up to 3000 tuples/run. Keep only sName (10 bytes).  $bf_{\text{sName}} = \lfloor 800/10 \rfloor = 80$  tuples. Create totally  $\lceil 1334/100 \rceil = 14$  sorted runs. Write  $\lceil 3000/80 \rceil = 38$  pages for 13 sorted runs and  $\lceil 1000/80 \rceil = 13$  pages for 1 sorted run. Totally write  $(38 \times 13) + 13 = 507$  pages.

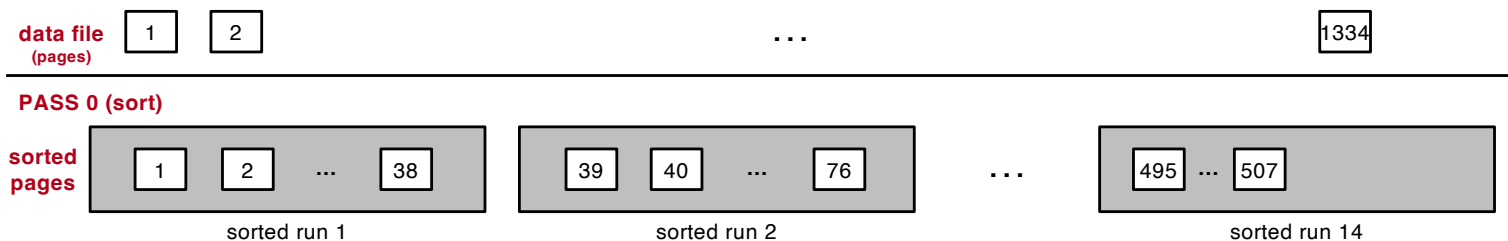
Sort page I/O cost: 1,334 pages read + 507 pages written = 1,841.

**Merge:** Read and merge the 500 sorted pages using a 14-way merge.

Merge page I/O cost: 507

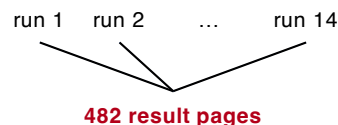
Projection page I/O cost:  $1841 + 507 = \underline{2,348}$

Query result pages:  $\lceil 40000 \text{ tuples} \times .95 / \lfloor 800/10 \rfloor \text{ tuples/page} \rceil = \underline{475}$  since it is assumed that 5% of sailor names are the same.



Since there are 100 buffer pages, for each sorting run read into the buffer 100 Sailor pages but write  $\lceil 3000 \text{ tuples} / \lfloor 800/10 \rfloor \text{ tuples/page} \rceil = 38$  pages since only sName is kept (as the other attributes are not needed). There are  $\lceil 1334/100 \rceil = 13$  full sorted runs created each of size 38 pages and 1 sorted run of  $\lceil 1000 \text{ tuples} / \lfloor 800/10 \rfloor \text{ tuples/page} \rceil = 13$  pages.

**PASS 1 (merge)**



## EXERCISE 5

Use the following information about the relations to estimate the page I/O cost to compute the query result using the stated join strategies.

Sailor(sailorId: 2 bytes, sName: 10 bytes, hkid: 10 bytes, rating: 2 bytes, age: 2 bytes)

Reserves(sailorId: 2 bytes, boatId: 2 bytes, rDate: 7 bytes)

**Query:** `select *  
from Sailor natural join Reserves;`

Page size: 1000 bytes; buffer size  $M$ : 100 pages; pointer size: 4 bytes.

Each Sailor tuple is 26 bytes, and each Reserves tuple is 11 bytes.

$n_{\text{Sailor}}$ : 10,000 tuples;  $bf_{\text{Sailor}}$ :  $\lfloor 1000 / 26 \rfloor = 38$  tuples per page.

$n_{\text{Reserves}}$ : 40,000 tuples;  $bf_{\text{Reserves}}$ :  $\lfloor 1000 / 11 \rfloor = 90$  tuples per page.

$B_{\text{Sailor}}$ :  $\lceil 10,000 / 38 \rceil = \lceil 263.15 \rceil = 264$  pages.

$B_{\text{Reserves}}$ :  $\lceil 40,000 / 90 \rceil = \lceil 444.44 \rceil = 445$  pages.

Since there are 10,000 Sailor tuples and 40,000 Reserves tuples, a sailor has, on average, 4 reservations.

```
select *  
from Sailor natural join Reserves;
```

$n_{\text{Sailor}}$ : 10,000 tuples  
 $bf_{\text{Sailor}}$ : 38 tuples/page  
 $B_{\text{Sailor}}$ : 264 pages  
 $n_{\text{Reserves}}$ : 40,000 tuples  
 $bf_{\text{Reserves}}$ : 90 tuples/page  
 $B_{\text{Reserves}}$ : 445 pages  
Buffer pages  $M$ : 100

## EXERCISE 5 (CONTD)

a) query page I/O cost for optimized block nested-loop join

i. using Sailor as the outer relation:  $\lceil B_r / (M-2) \rceil * B_s + B_r$

Read 98 pages of the Sailor relation into the buffer at a time (there are  $\lceil 264 / 98 \rceil = \lceil 2.69 \rceil = 3$  “blocks” of Sailor pages). (*Requires 98 buffer pages*)

For each “block” of Sailor pages we scan the Reserves relation (page-by-page) to find matching tuples. (*Requires 1 buffer page*)

Write matching results to the output. (*Requires 1 buffer page*)

Query page I/O cost:  $3 * 445 + 264 = 1,599$

ii. using Reserves as the outer relation:  $\lceil B_r / (M-2) \rceil * B_s + B_r$

Read 98 pages of the Reserves relation into the buffer at a time (there are  $\lceil 445 / 98 \rceil = \lceil 4.54 \rceil = 5$  “blocks” of Reserves pages). (*Requires 98 buffer pages*)

For each “block” of Reserves pages we scan the Sailor relation (page-by-page) to find matching tuples. (*Requires 1 buffer page*)

Write matching results to the output. (*Requires 1 buffer page*)

Query page I/O cost:  $5 * 264 + 445 = 1,765$

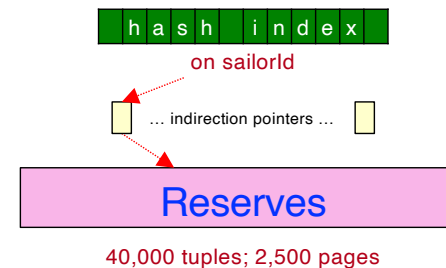
```
select *
from Sailor natural join Reserves;
```

## EXERCISE 5 (CONTD)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $bf_{\text{Sailor}}$ : 38 tuples/page  
 $B_{\text{Sailor}}$ : 264 pages  
 $n_{\text{Reserves}}$ : 40,000 tuples  
 $bf_{\text{Reserves}}$ : 90 tuples/page  
 $B_{\text{Reserves}}$ : 445 pages  
 Buffer pages  $M$ : 100

b) query page I/O cost for indexed nested-loop join with hash index on Reserves.sailorId (assume no overflow).

For each **Sailor** tuple, find the corresponding entry in the hash index on **Reserves.sailorId**. This takes **1 page I/O per Sailor tuple** (since we assume no overflow).



Since each **Sailor** tuple has on average **4 reservations**, and, since the hash index is non-clustering (secondary), we expect each **sailorId** to have **4** matching tuples in the **Reserves** relation on average. Thus, there is an indirection level, and we need **1 page I/O per sailorId** to retrieve the indirection pointers.

We note that these **4** indirection pointers occupy (**4** pointers \* **4** bytes per pointer) = **16** bytes, which easily fit on one page.

Finally, we need **4 page I/Os per sailorId** to retrieve the corresponding **Reserves** records (i.e., **1 page I/O** for each indirection pointer).

Query page I/O cost: cost of reading **Sailor** + (# tuples in **Sailor** \* (**1** for hash index + **1** for indirection pointers + **4** for **Reserves** records))  
 $= 264 + 10,000 * 6 = \underline{60,264}$

```
select *
from Sailor natural join Reserves;
```

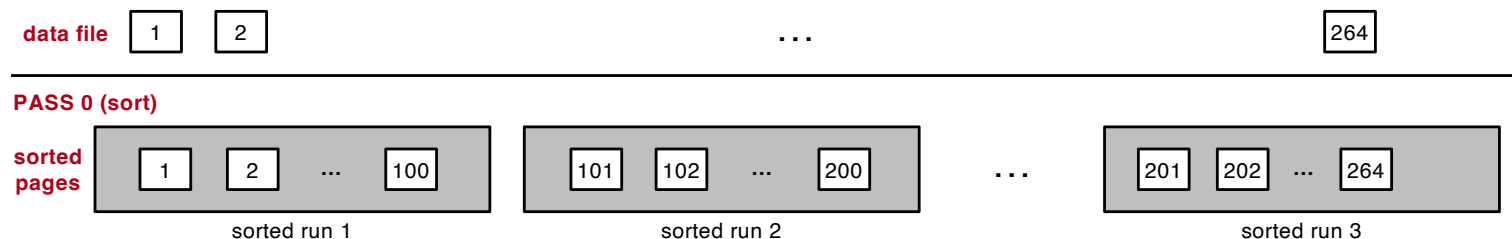
$n_{\text{Sailor}}$ : 10,000 tuples  
 $bf_{\text{Sailor}}$ : 38 tuples/page  
 $B_{\text{Sailor}}$ : 264 pages  
 $n_{\text{Reserves}}$ : 40,000 tuples  
 $bf_{\text{Reserves}}$ : 90 tuples/page  
 $B_{\text{Reserves}}$ : 445 pages  
 Buffer pages  $M$ : 100

## EXERCISE 5 (CONTD)

c) query page I/O cost for merge join (assume that both relations are not sorted initially)

Sort cost (to sort the relations using external sorting)

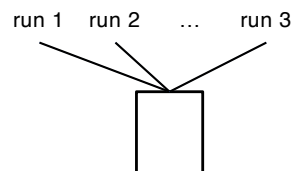
Cost to sort Sailor on sailord



Since there are 100 buffer pages, for each sorting run read into the buffer up to 100 Sailor pages and write up to 100 pages. There are  $\lceil 264/100 \rceil = 3$  sorted runs created, 2 of size 100 pages and 1 of size 64 pages. Totally  $264 \times 2 = 528$  pages are read and written.

PASS 1 (merge)

The Sailor relation can be merged in 1 pass since only 3 buffer pages are required to merge the 3 sorted runs. Totally  $264 \times 2 = 528$  pages are read and written.



Page I/O cost to sort Sailor:  $\underset{r}{264} + \underset{w}{264}$  (Pass 0) +  $\underset{r}{264} + \underset{w}{264}$  (Pass 1) = 1,056

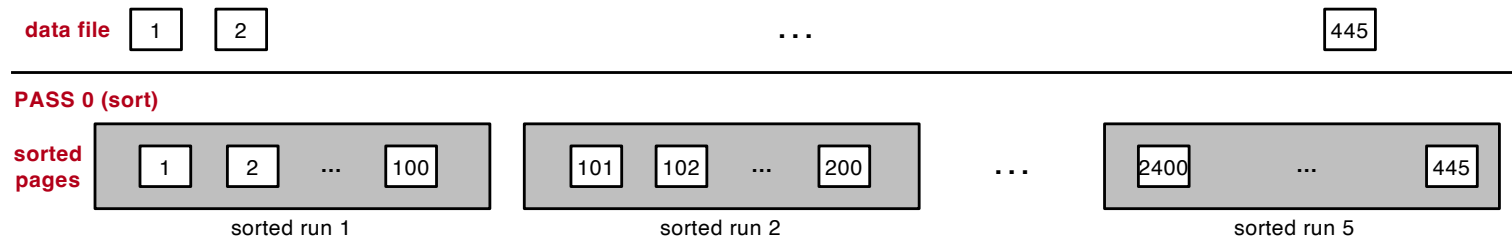


```
select *
from Sailor natural join Reserves;
```

## EXERCISE 5 (CONTD)

$n_{\text{Sailor}}$ : 10,000 tuples  
 $bf_{\text{Sailor}}$ : 38 tuples/page  
 $B_{\text{Sailor}}$ : 264 pages  
 $n_{\text{Reserves}}$ : 40,000 tuples  
 $bf_{\text{Reserves}}$ : 90 tuples/page  
 $B_{\text{Reserves}}$ : 445 pages  
 Buffer pages  $M$ : 100

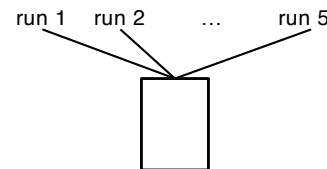
### Cost to sort Reserves on sailord



Since there are 100 buffer pages, for each sorting run read into the buffer up to 100 Reserves pages and write up to 100 pages. There are  $\lceil 445/100 \rceil = 5$  sorted runs created, 4 of size 100 pages and 1 of size 45 pages. Totally  $445 \times 2 = 890$  pages are read and written.

#### PASS 1 (merge)

The Reserves relation can be merged in 1 pass since only 5 buffer pages are required to merge the 5 sorted runs. Totally  $445 \times 2 = 890$  pages are read and written.



Page I/O cost to sort Reserves:  $\underset{r}{445} + \underset{w}{445}$  (Pass 0) +  $\underset{r}{445} + \underset{w}{445}$  (Pass 1) = 1,780

Total page I/O cost to sort the relations:  $1056 + 1780 = \underline{2,836}$

Merge cost (to join the relations)

Read both sorted relations and join during merge.

Total page I/O cost to join the relations:  $264 + 445 = \underline{709}$

Query page I/O cost:  $2,836 + 709 = \underline{3,545}$

## EXERCISE 6

Use the following information about the relations to estimate the page I/O cost to compute the query result.

Student(studentId: 8 bytes, firstName: 20 bytes, lastName: 25 bytes, email: 15 bytes, unitId: 4 bytes)

EnrollsIn(studentId: 8 bytes, courseId: 8 bytes, grade: 4 bytes)

The Student relation contains 10,000 tuples in 1,000 pages and the EnrollsIn relation contains 50,000 tuples in 1,352 pages. The page size is 1000. There are 25 different academic units and 1,000 different courses.

Our goal is to process the query:

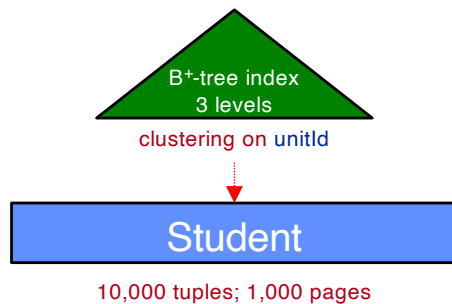
```
select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';
```

### Some useful statistics

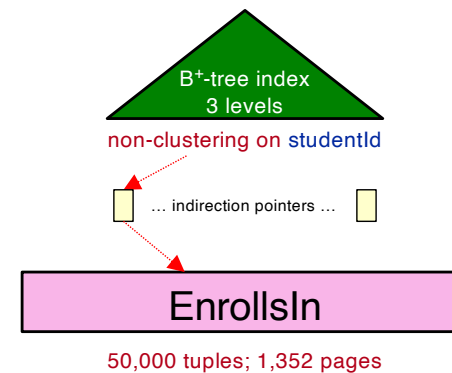
- On average, a student enrolls in  $(50,000/10,000) = 5$  courses.
- On average, a department has  $(10,000/25) = 400$  students.
- On average, each course has an enrollment of  $(50,000/1,000) = 50$  students.

## EXERCISE 6 (CONT'D)

The Student relation contains a clustering index on unitId and the EnrollsIn relation contains a non-clustering index on studentId. Each available index is a B<sup>+</sup>-tree with 3 levels. For non-clustering indexes, each pointer leads to a different page.



```
select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';
```



### Some useful statistics

- On average, a student enrolls in 5 courses.
- On average, a department has 400 students.
- On average, each course has an enrollment of 50 students.

Estimate the minimum page I/O cost of processing the query using a fully pipelined execution method (i.e., do not materialize anything except the query result).

```
select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';
```

```
Student(studentId, firstName, lastName, email, unitId)
EnrollsIn(studentId, courseId, grade)
```

## EXERCISE 6 (CONT'D)

$n_{\text{Student}}$ : 10,000 tuples  
 $bf_{\text{Student}}$ : 10 tuples/page  
 $B_{\text{Student}}$ : 1,000 pages  
 $n_{\text{EnrollsIn}}$ : 50,000 tuples  
 $bf_{\text{EnrollsIn}}$ : 37 tuples/page  
 $B_{\text{EnrollsIn}}$ : 1,352 pages  
 EnrollsIn tuples/student: 5  
 Number of academic units: 25  
 Number of courses: 1,000  
 Page size: 1,000

### Base Strategy

Do not use any indexes, join **Student** and **EnrollsIn** using block nested-loop and check the conditions `courseId='COMP 3311'` and `unitId='COMP'` in the buffer. Since the buffer size is unknown, assume the minimum buffer size (3 pages) and so use the worst-case join cost.

Page I/O cost:  $1,000 * 1,352 + 1,000 = \underline{1,353,000}$

a) Cost to evaluate  $\sigma_{\text{courseId}='COMP 3311'}$  EnrollsIn (*no index on courseId*)

**Strategy a1:** linear search on EnrollsIn

Read **EnrollsIn** page-by-page requiring **1,352 page I/Os** and check the condition `courseId='COMP 3311'` in each EnrollsIn tuple.

The result will occupy  $\lceil 50 \text{ tuples per course} / 37 \text{ tuples/page} \rceil = 2$  pages which are kept in the buffer.

Page I/O cost: 1,352

**Strategy a2:** perform the selection in the buffer (*on-the-fly*) during the join

The condition `courseId='COMP 3311'` is checked in the buffer for each result tuple of the join operation.

Page I/O cost: 0

```
select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';
```

```
Student(studentId, firstName, lastName, email, unitId)
EnrollsIn(studentId, courseId, grade)
```

## EXERCISE 6 (CONT'D)

$n_{\text{Student}}$ : 10,000 tuples  
 $bf_{\text{Student}}$ : 10 tuples/page  
 $B_{\text{Student}}$ : 1,000 pages  
 $n_{\text{EnrollsIn}}$ : 50,000 tuples  
 $bf_{\text{EnrollsIn}}$ : 37 tuples/page  
 $B_{\text{EnrollsIn}}$ : 1,352 pages  
 EnrollsIn tuples/student: 5  
 Number of academic units: 25  
 Number of courses: 1,000  
 Page size: 1,000

b) Cost to evaluate  $\sigma_{\text{unitId}='COMP'} \text{Student}$

**Strategy b1:** linear search on Student

Read Student page-by-page requiring 1,000 page I/Os and check the condition  $\text{unitId}='COMP'$  in each Student tuple.

Page I/O cost: 1,000

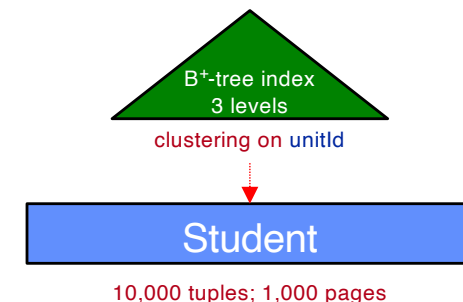
**Strategy b2:** index lookup using B<sup>+</sup>-tree index on Student.unitId

The selection reads 3 index pages using the clustering index on unitId.

Since the file is ordered on unitId, there are 25 different academic units and students are distributed uniformly over academic units,  $\lceil 1,000 / 25 \rceil = 40$  pages of Student tuples are read.

Page I/O cost: 3 + 40 = 43

For both strategies, 400 Student tuples will qualify for which only the studentId, firstName and lastName (total 53 bytes) are kept. The result will occupy  $\lceil 400 / \lfloor 750 / (53) \rfloor \rceil = 29$  pages which are kept in the buffer.



```

select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';

```

Student(studentId, firstName, lastName, email, unitId)  
 EnrollsIn(studentId, courseId, grade)

## EXERCISE 6 (CONT'D)

$n_{\text{Student}}$ : 10,000 tuples  
 $bf_{\text{Student}}$ : 10 tuples/page  
 $B_{\text{Student}}$ : 1,000 pages  
 $n_{\text{EnrollsIn}}$ : 50,000 tuples  
 $bf_{\text{EnrollsIn}}$ : 37 tuples/page  
 $B_{\text{EnrollsIn}}$ : 1,352 pages  
 EnrollsIn tuples/student: 5  
 Number of academic units: 25  
 Number of courses: 1,000  
 Page size: 1,000

### c) Cost to evaluate Student $\bowtie$ EnrollsIn

**Strategy c1:** block nested-loop — after **Strategy a2**

Read 1,352 pages of EnrollsIn and join with Student which is in the buffer.

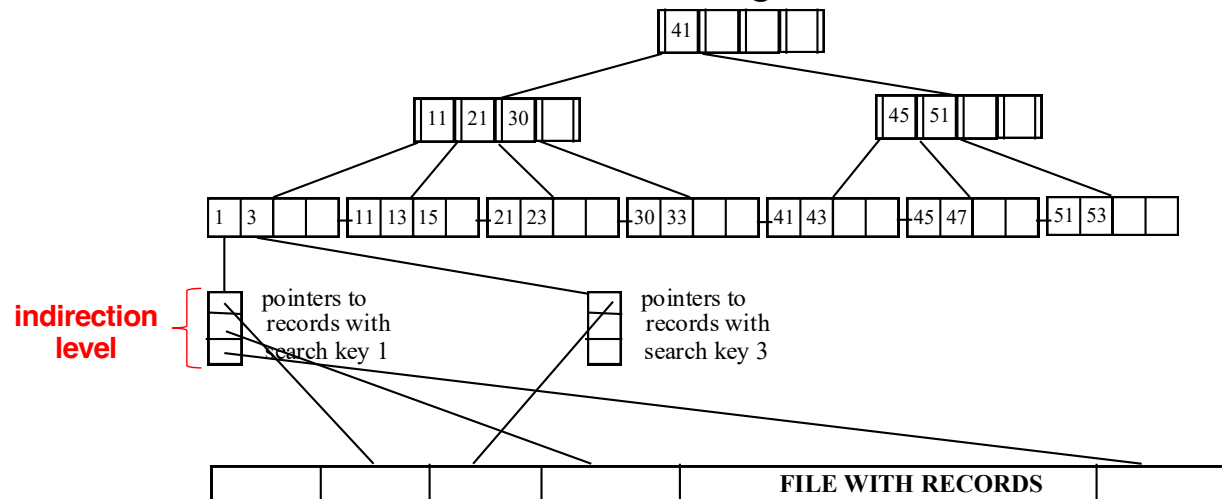
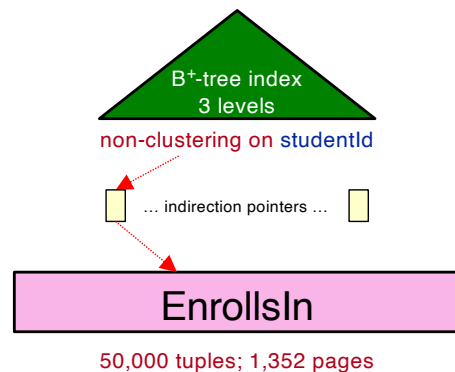
Page I/O cost: 1,352

**Strategy c2:** indexed nested-loop using studentId index on EnrollsIn

For each of the 400 students in the selection result  $\sigma_{\text{unitId}='COMP'}\text{Student}$ , use the studentId index on EnrollsIn to find the corresponding EnrollsIn tuples.

For each studentId (student) retrieve  $5/2 = 2.5$  EnrollsIn tuples, on average.

Since the studentId index on EnrollsIn is non-clustering, the record pointers to the EnrollsIn records first need to be retrieved as shown in the figure below.



```
select firstName, lastName
from Student natural join EnrollsIn
where courseId='COMP 3311'
and unitId='COMP';
```

```
Student(studentId, firstName, lastName, email, unitId)
EnrollsIn(studentId, courseId, grade)
```

## EXERCISE 6 (CONT'D)

$n_{\text{Student}}$ : 10,000 tuples  
 $bf_{\text{Student}}$ : 10 tuples/page  
 $B_{\text{Student}}$ : 1,000 pages  
 $n_{\text{EnrollsIn}}$ : 50,000 tuples  
 $bf_{\text{EnrollsIn}}$ : 37 tuples/page  
 $B_{\text{EnrollsIn}}$ : 1,352 pages  
 EnrollsIn tuples/student: 5  
 Number of academic units: 25  
 Number of courses: 1,000  
 Page size: 1,000

Assuming the 2.5 record pointers per student all fit on a page (which is highly likely), 400 page I/Os are needed to retrieve the indirection record pointers (one for each student).

For each EnrollsIn tuple in the join result, check if it meets the condition `courseId='COMP 3311'`.

Page I/O cost:  $400 * (3 + 1 + 2.5) = \underline{2,600}$

**Strategy c3:** block nested-loop (*in the buffer*) — after **Strategy a1**

The join can be done in the buffer since the result of a), 2 pages, and b), 29 pages, are in the buffer.

Page I/O cost: 0

d) Cost to evaluate  $\pi_{\text{firstName, lastName}}$

**Strategy d1:** perform the projection in the buffer (*on-the-fly*) during the join

The projection is done in the buffer for each result tuple of the join operation.

Page I/O cost: 0

## EXERCISE 6 (CONT'D)

### Minimum query processing page I/O cost

|                                |              |
|--------------------------------|--------------|
| Strategy a1: linear search     | = 1,352      |
| Strategy b2: index lookup      | = 43         |
| Strategy c3: block nested-loop | = 0          |
| Strategy d1: on-the fly        | = 0          |
| Total                          | <u>1,395</u> |

### Minimum query processing page I/O cost

|                                |              |
|--------------------------------|--------------|
| Strategy a2: on-the-fly        | = 0          |
| Strategy b2: index lookup      | = 43         |
| Strategy c1: block nested-loop | = 1,352      |
| Strategy d1: on-the fly        | = 0          |
| Total                          | <u>1,395</u> |

### Alternative query processing page I/O cost

|                                |              |
|--------------------------------|--------------|
| Strategy a2: on-the-fly        | = 0          |
| Strategy b2: index lookup      | = 43         |
| Strategy c2: index nested-loop | = 2,600      |
| Strategy d1: on-the fly        | = 0          |
| Total                          | <u>2,643</u> |

**Recall:** The cost to write the query result is never included in the cost estimate.



# EXERCISE 6 (CONT'D)

## Minimum Query Processing Page I/O Cost Query Evaluation Trees

